LAPORAN PRAKTIKUM
DATA INGESTION – EVENT-DRIVEN & STREAMING
MK PRAKTIKUM DATA PROCESSING WORKFLOW

Dosen Pengampu : Isbat Uzzin Nadhori S.Kom., M.T



Disusun Oleh:

Kadek Savita Dyutianaya

NRP. 3324600033

Sains Data Terapan B

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER

PROGRAM STUDI SAINS DATA TERAPAN

2026

A. EKSPERIMEN & ANALISIS

1. Verifikasi Environment

Tujuan : Memastikan environment praktikum siap digunakan sebelum menjalankan Kafka.

Langkah 1: Verifikasi Docker

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> docker --version
Docker version 28.5.1, build e180ab8
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> docker compose version
Docker Compose version v2.40.0-desktop.1
```

Langkah 2: Verifikasi Python

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> python --version
Python 3.10.11
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> python -c "import confluent_kafka; print(confluent_kafka.version())"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'confluent_kafka'
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> pip install -r requirements.txt
Collecting confluent-kafka==2.15.0 (from -r requirements.txt (line 1))
  Downloading confluent_kafka-2.15.0-cp310-cp310-win_amd64.whl.metadata (34 kB)
Requirement already satisfied: typing-extensions in c:\users\putuw\appdata\local\programs\python\python310\lib\site-packages (from confluent-kafka==2.15.0->-
r requirements.txt (line 1)) (4.16.0)
Downloading confluent_kafka-2.15.0-cp310-cp310-win_amd64.whl (4.5 MB)
   ---------------------------------------- 4.5/4.5 MB 0.5 MB/s  0:00:08
Installing collected packages: confluent-kafka
Successfully installed confluent-kafka-2.15.0
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> python -c "import confluent_kafka; print(confluent_kafka.version())"
2.15.0
```

2. Menjalankan Message Broker

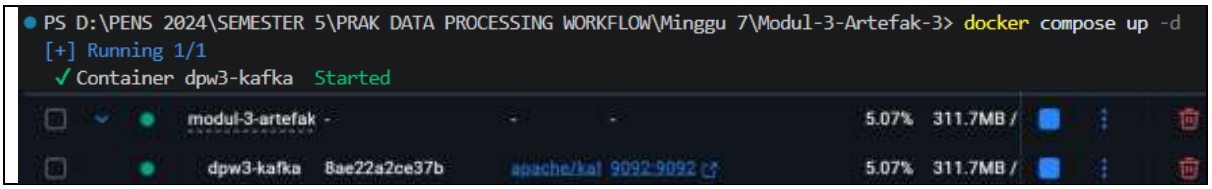
Tujuan : Menjalankan Apache Kafka sebagai message broker menggunakan Docker Compose.

Langkah 1: Build

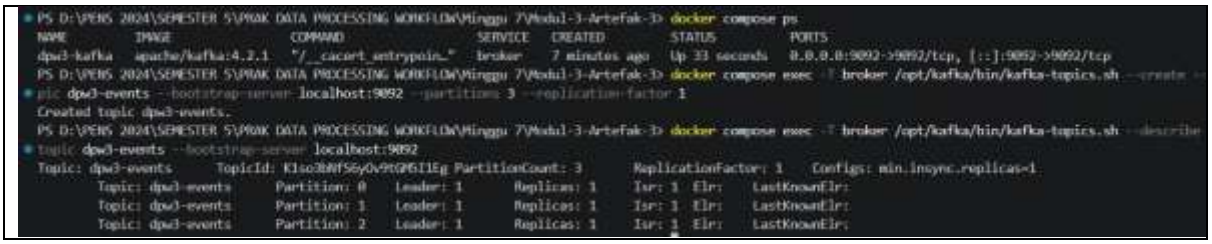
```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> docker compose config
name: modul-3-artefak-3
services:
  broker:
    container_name: dpw3-kafka
    environment:
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@localhost:9093
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: "0"
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_NODE_ID: "1"
      KAFKA_NUM_PARTITIONS: "3"
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: "1"
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: "1"
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: "1"
    image: apache/kafka:4.2.1
    networks:
      default: null
    ports:
      - mode: ingress
        target: 9092
        published: "9092"
        protocol: tcp
networks:
  default:
    name: modul-3-artefak-3_default
```

Langkah 2: Run

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> docker compose up -d
[+] Running 1/1
 ✓ Container dpw3-kafka  Started
```



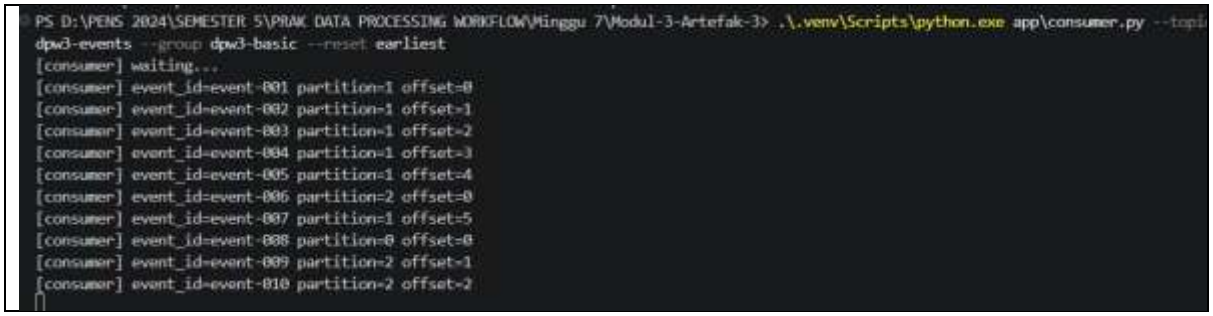
Langkah 3: Buat Topic



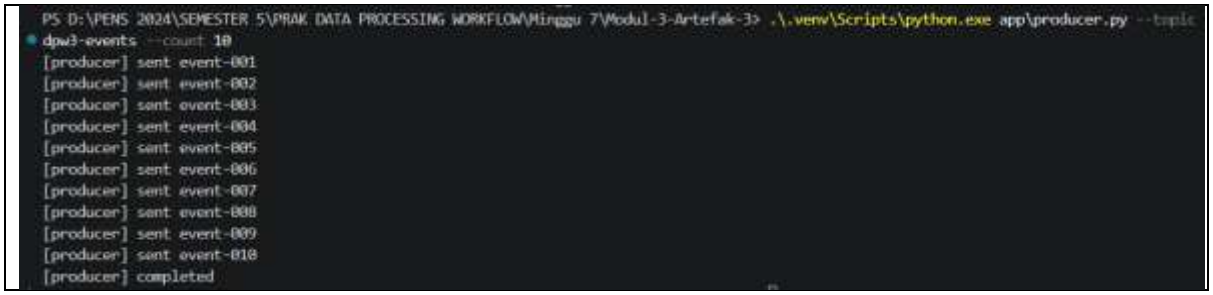
3. Producer–Consumer Asynchronous

Tujuan : Menjalankan producer dan consumer sebagai proses terpisah dan mengamati aliran event melalui Kafka.

Langkah 1: Run Consumer (hasil after run langkah 2)



Langkah 2: Run Producer

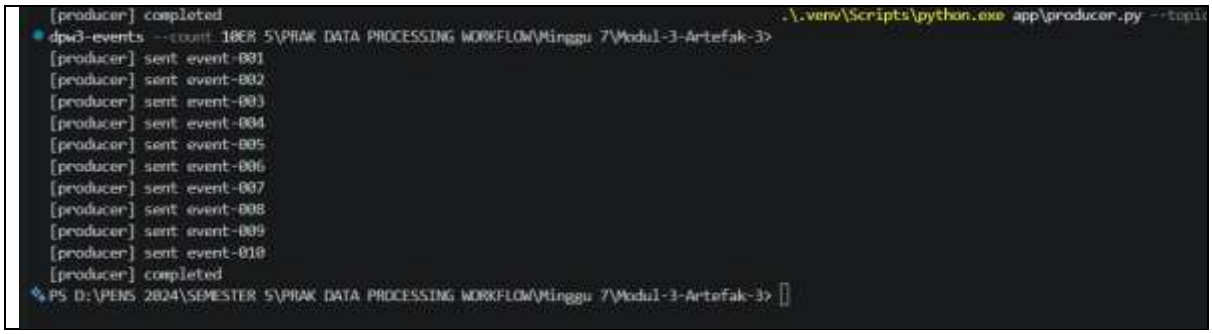


Pertanyaan analisis

Apakah producer harus menunggu consumer selesai memproses event sebelum producer dapat mengirim event berikutnya?

= Tidak. Producer mengirim event ke Kafka sebagai perantara, sehingga producer dapat mengirim event berikutnya tanpa menunggu consumer selesai memproses event sebelumnya. Consumer membaca dan memproses event secara terpisah.

Failure Scenario: cntrl+c lalu jalankan producer



Hasilnya tetap jalan, producer tetap dapat mengirim event saat consumer tidak aktif. Artinya benar bahwa producer dan consumer tidak harus berjalan sebagai satu proses yang sama. Kafka menjadi perantara dan distribusi event

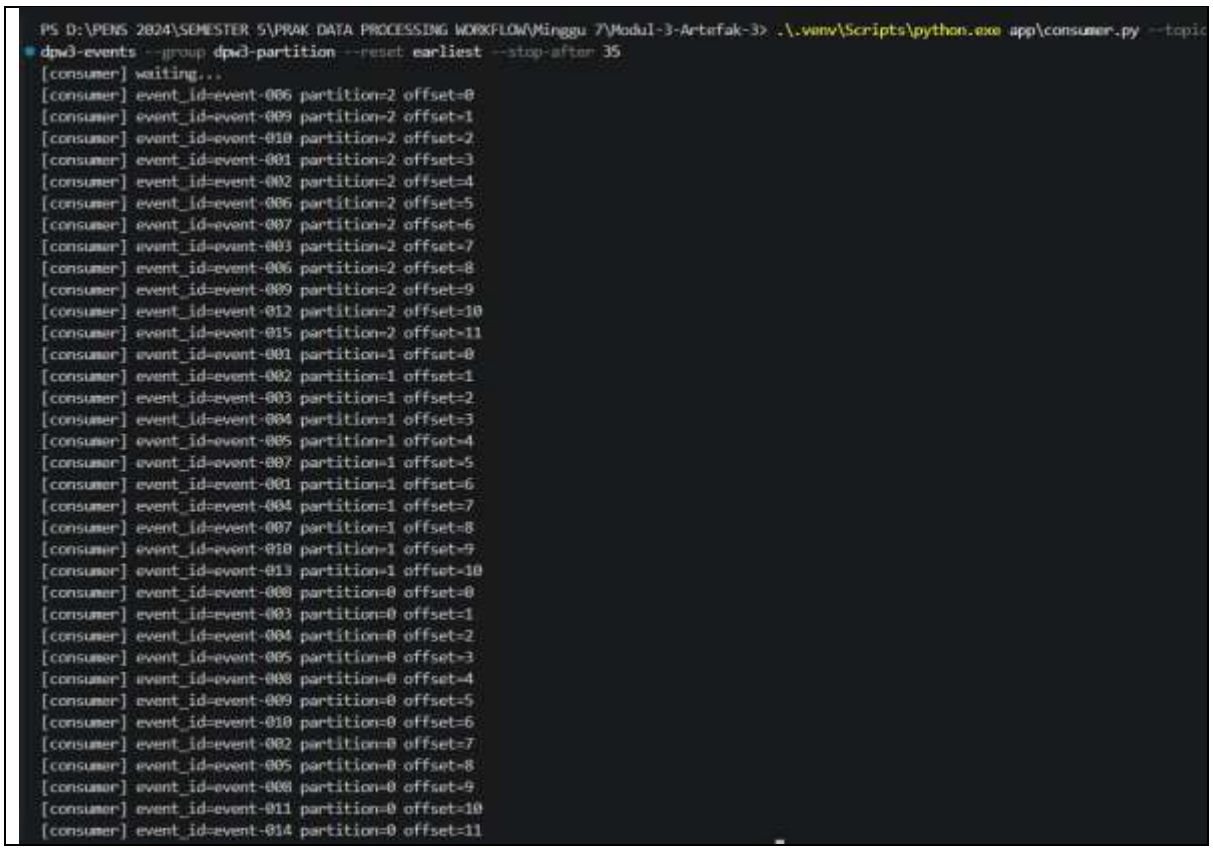
4. **Eksperimen Topic dan Partition**

Tujuan : Mengamati bagaimana event didistribusikan ke beberapa partition dalam satu topic.

Langkah 1: Produce



Langkah 2: Consume



Buat catatan sederhana:

event_id	partition	offset
event-001	1	0
event-001	1	6
event-001	2	3
event-002	0	7
event-002	1	1

event-002	2	4
event-003	0	1
event-003	1	2
event-003	2	7
event-004	0	2
event-004	1	3
event-004	1	7
event-005	0	3
event-005	0	8
event-005	1	4
event-006	2	0
event-006	2	5
event-006	2	8
event-007	1	5
event-007	1	8
event-007	2	6
event-008	0	0
event-008	0	4
event-008	0	9
event-009	0	5
event-009	2	1
event-009	2	9
event-010	0	6
event-010	1	9
event-010	2	2
event-011	0	10
event-012	2	10
event-013	1	10
event-014	0	11
event-015	2	11

Pertanyaan analisis:

Pertanyaan 1:

Mengapa event dapat berada pada partition yang berbeda?

= Event dapat berada pada partition berbeda karena Kafka membagi record ke beberapa partition, biasanya berdasarkan hash key yang dikirim producer. Dalam percobaan ini key-nya entity_id, jadi event dengan entity berbeda dapat masuk ke partition berbeda.

Pertanyaan 2:

Apakah offset 5 pada partition 0 sama dengan offset 5 pada partition 1?

= Tidak. Offset 5 pada partition 0 dan offset 5 pada partition 1 menunjukkan posisi record yang berbeda, karena offset dihitung secara terpisah untuk setiap partition.

Verificiation

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> docker compose exec -T broker /opt/kafka/bin/kafka
get-offsets.sh --bootstrap-server localhost:9092 --topic dpw3-events
dpw3-events:0:12
dpw3-events:1:11
dpw3-events:2:12
```

5. Eksperimen Consumer Group

Tujuan : Mengamati bagaimana partition dibagikan kepada beberapa consumer dalam consumer group yang sama.

Langkah 1: Consumer 1

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> .\.venv\Scripts\python.exe app\consumer.py --topi
dpw3-events --group dpw3-group --consumer-name C1
[C1] waiting...
```

Langkah 2: Consumer 2

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> .\.venv\Scripts\python.exe app\consumer.py --topi
dpw3-events --group dpw3-group --consumer-name C2
[C2] waiting...
```

Langkah 3: Produce

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> .\.venv\Scripts\python.exe app\producer.py --topi
dpw3-events --count 20
[producer] sent event-001
[producer] sent event-002
[producer] sent event-003
[producer] sent event-004
[producer] sent event-005
[producer] sent event-006
[producer] sent event-007
[producer] sent event-008
[producer] sent event-009
[producer] sent event-010
[producer] sent event-011
[producer] sent event-012
[producer] sent event-013
[producer] sent event-014
[producer] sent event-015
[producer] sent event-016
[producer] sent event-017
[producer] sent event-018
[producer] sent event-019
[producer] sent event-020
[producer] completed
```

Hasil C1

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> .\.venv\Scripts\python.exe app\consumer.py --topic
dpw3-events --group dpw3-group --consumer-name C1
[C1] waiting...
[C1] event_id=event-003 partition=0 offset=12
[C1] event_id=event-005 partition=1 offset=11
[C1] event_id=event-006 partition=0 offset=13
[C1] event_id=event-008 partition=0 offset=14
[C1] event_id=event-009 partition=0 offset=15
[C1] event_id=event-010 partition=1 offset=12
[C1] event_id=event-012 partition=1 offset=13
[C1] event_id=event-013 partition=1 offset=14
[C1] event_id=event-014 partition=1 offset=15
[C1] event_id=event-015 partition=0 offset=16
[C1] event_id=event-016 partition=1 offset=16
[C1] event_id=event-017 partition=1 offset=17
[C1] event_id=event-018 partition=0 offset=17
[C1] event_id=event-019 partition=0 offset=18
[C1] event_id=event-020 partition=0 offset=19
```

Hasil C2

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> .\.venv\Scripts\python.exe app\consumer.py --topic
dpw3-events --group dpw3-group --consumer-name C2
[C2] waiting...
[C2] event_id=event-001 partition=2 offset=12
[C2] event_id=event-002 partition=2 offset=13
[C2] event_id=event-004 partition=2 offset=14
[C2] event_id=event-007 partition=2 offset=15
[C2] event_id=event-011 partition=2 offset=16
```

Hasil pembagian partition

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> docker compose exec -T broker /opt/kafka/bin/kafka-
consumer-groups.sh --bootstrap-server localhost:9092 --describe --group dpw3-group

GROUP           TOPIC          PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID                                 HOST
            CLIENT-ID
dpw3-group      dpw3-events    0          -               20              -               C1-01ba10de-7ccb-423d-9445-b9909d76e955 /172.
24.0.1      C1
dpw3-group      dpw3-events    1          -               18              -               C1-01ba10de-7ccb-423d-9445-b9909d76e955 /172.
24.0.1      C1
dpw3-group      dpw3-events    2          -               17              -               C2-a9e68b47-ece6-47d5-b643-654af7e824d1 /172.
24.0.1      C2
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3>
```

Langkah 4: Failure

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> docker compose exec -T broker /opt/kafka/bin/kafk
consumer-groups.sh --bootstrap-server localhost:9092 --describe --group dpw3-group

GROUP           TOPIC          PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID                                 HOS
            CLIENT-ID
dpw3-group      dpw3-events    0          -               20              -               C1-01ba10de-7ccb-423d-9445-b9909d76e955 /17
24.0.1      C1
dpw3-group      dpw3-events    1          -               18              -               C1-01ba10de-7ccb-423d-9445-b9909d76e955 /17
24.0.1      C1
dpw3-group      dpw3-events    2          -               17              -               C1-01ba10de-7ccb-423d-9445-b9909d76e955 /17
24.0.1      C1
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> .\.venv\Scripts\python.exe app\producer.py --topi
dpw3-events --count 5
[producer] sent event-001
[producer] sent event-002
[producer] sent event-003
[producer] sent event-004
[producer] sent event-005
[producer] completed
```

Hasil C1 setelah C2 dihentikan

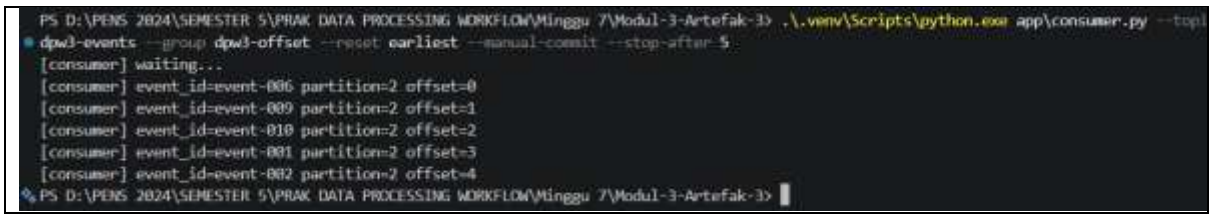
```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> .\.venv\Scripts\python.exe app\consumer.py --topic
dpw3-events --group dpw3-group --consumer-name C1
[C1] waiting...
[C1] event_id=event-003 partition=0 offset=12
[C1] event_id=event-005 partition=1 offset=11
[C1] event_id=event-006 partition=0 offset=13
[C1] event_id=event-008 partition=0 offset=14
[C1] event_id=event-009 partition=0 offset=15
[C1] event_id=event-010 partition=1 offset=12
[C1] event_id=event-012 partition=1 offset=13
[C1] event_id=event-013 partition=1 offset=14
[C1] event_id=event-014 partition=1 offset=15
[C1] event_id=event-015 partition=0 offset=16
[C1] event_id=event-016 partition=1 offset=16
[C1] event_id=event-017 partition=1 offset=17
[C1] event_id=event-018 partition=0 offset=17
[C1] event_id=event-019 partition=0 offset=18
[C1] event_id=event-020 partition=0 offset=19
[C1] event_id=event-001 partition=0 offset=20
[C1] event_id=event-002 partition=0 offset=21
[C1] event_id=event-003 partition=2 offset=17
[C1] event_id=event-004 partition=1 offset=18
[C1] event_id=event-005 partition=0 offset=22
```

Kesimpulannya: Kafka membagi partition di antara consumer dalam satu group. Ketika C2 berhenti, Kafka melakukan rebalance dan partition yang ditanganinya dapat dialihkan ke consumer aktif yang tersisa. (event-011 hilang karena merupakn C2)

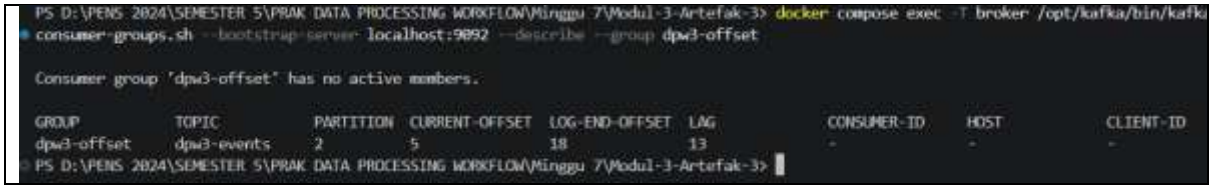
6. **Eksperimen Offset**

Tujuan: Mengamati fungsi committed offset sebagai posisi konsumsi consumer group.

Langkah 1: Consume



Langkah 2: Inspect Offset



Langkah 3: Run Again



Consumer melanjutkan konsumsi berdasarkan posisi yang telah disimpan oleh consumer group.

Pertanyaan analisis:

Mengapa consumer tidak kembali membaca seluruh event dari awal?

= Consumer tidak membaca ulang semua event karena consumer group menyimpan committed offset sebagai posisi terakhir yang sudah dikonsumsi. Saat dijalankan kembali dengan group.id yang sama, consumer melanjutkan dari offset tersimpan, bukan dari awal.

Verification

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> docker compose exec -T broker /opt/kafka/bin/kafka-
consumer-groups.sh --bootstrap-server localhost:9092 --describe --group dpw3-offset

Consumer group 'dpw3-offset' has no active members.

GROUP          TOPIC          PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID     HOST            CLIENT-ID
dpw3-offset    dpw3-events    2          10              18              8               -               -               -
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3>
```

Terdapat perubahan pada CURRENT-OFFSET dan LAG, di awal 5 dan 13, sekarang 10 dan 8

7. Failure Injection

Tujuan : Membuktikan bahwa kegagalan consumer dapat menyebabkan perubahan assignment dan consumer group dapat melanjutkan konsumsi.

Langkah 1: Consumer 1

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> .\.venv\Scripts\python.exe app\consumer.py --topic
dpw3-events --group dpw3-failure --consumer-name C1 --fail-after 3
[C1] waiting...
```

Langkah 2: Consumer 2

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> Set-Location "D:\PENS 2024\SEMESTER 5\PRAK DATA PRO
CESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3"
>> .\.venv\Scripts\python.exe app\consumer.py --topic dpw3-events --group dpw3-failure --consumer-name C2
[C2] waiting...
```

Langkah 3: Produce

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> .\.venv\Scripts\python.exe app\producer.py --topic
dpw3-events --count 10
[producer] sent event-001
[producer] sent event-002
[producer] sent event-003
[producer] sent event-004
[producer] sent event-005
[producer] sent event-006
[producer] sent event-007
[producer] sent event-008
[producer] sent event-009
[producer] sent event-010
[producer] completed
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3>
```

Hasil C1

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> .\.venv\Scripts\python.exe app\consumer.py --topic
dpw3-events --group dpw3-failure --consumer-name C1 --fail-after 3
[C1] waiting...
[C1] event_id=event-001 partition=1 offset=19
[C1] event_id=event-002 partition=1 offset=20
[C1] event_id=event-003 partition=1 offset=21
[consumer] FAILED: intentional failure
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3>
```

Hasil C2

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> Set-Location "D:\PENS 2024\SEMESTER 5\PRAK DATA PRO
CESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3"
>> .\.venv\Scripts\python.exe app\consumer.py --topic dpw3-events --group dpw3-failure --consumer-name C2
[C2] waiting...
[C2] event_id=event-005 partition=2 offset=18
[C2] event_id=event-006 partition=2 offset=19
```

Langkah 4: Observe Failure

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> docker compose exec -T broker /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --group dpw3-failure

GROUP          TOPIC          PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG  CONSUMER-ID                                    HOST            CLIENT-ID
dpw3-failure   dpw3-events    0          -               26              -    C2-6bd176ae-6953-46c5-8acb-3a5aa808892e /172.24.0.1     C2
dpw3-failure   dpw3-events    1          -               24              -    C2-6bd176ae-6953-46c5-8acb-3a5aa808892e /172.24.0.1     C2
dpw3-failure   dpw3-events    2          -               20              -    C2-6bd176ae-6953-46c5-8acb-3a5aa808892e /172.24.0.1     C2
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3>
```

Pemulihan:

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> docker compose exec -T broker /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --group dpw3-failure

GROUP          TOPIC          PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG  CONSUMER-ID                                    HOST            CLIENT-ID
dpw3-failure   dpw3-events    0          -               26              -    C2-6bd176ae-6953-46c5-8acb-3a5aa808892e /172.24.0.1     C2
dpw3-failure   dpw3-events    1          -               24              -    C2-6bd176ae-6953-46c5-8acb-3a5aa808892e /172.24.0.1     C2
dpw3-failure   dpw3-events    2          -               20              -    C2-6bd176ae-6953-46c5-8acb-3a5aa808892e /172.24.0.1     C2
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> .\.venv\Scripts\python.exe app\consumer.py --topic dpw3-events --group dpw3-failure --consumer-name C1-recovery
[C1-recovery] waiting...
```

Bukti consumer pemulihan mendapat event baru

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> .\.venv\Scripts\python.exe app\producer.py --topic dpw3-events --count 5
[producer] sent event-001
[producer] sent event-002
[producer] sent event-003
[producer] sent event-004
[producer] sent event-005
[producer] completed
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3>
```

8. At-least-once vs Exactly-once

Tujuan: Membuktikan perbedaan hasil At-least-once dan Exactly-once melalui failure yang terjadi pada saat pemrosesan. Eksperimen menggunakan dataset kecil sehingga hasil dapat dihitung dan dibandingkan.

a. eksperimen At-least-once

Langkah 1: Run

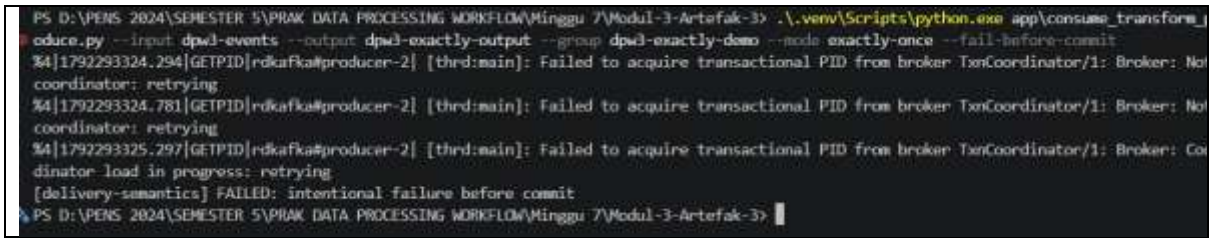
```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3> .\.venv\Scripts\python.exe app\consume_transform_produce.py --input dpw3-events --output dpw3-atleast-output --group dpw3-atleast-demo --mode atleast-once --fail-after-produce
[delivery-semantics] FAILED: intentional failure after output
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3>
```

Langkah 2: Restart



b. eksperimen exactly-once

Langkah 1: Run



Langkah 2: Restart



Langkah 3: Read Committed Output\



Perbandingan Hasil

Parameter	At-least-once	Exactly-once
Input event	10	10
Total output	11	10
Unique event	10	10
Duplicate output	1	0

Failure terjadi	Ya	Ya
Output transaction committed	Tidak	Ya

Pertanyaan Analisis

a. **Mengapa At-least-once dapat menghasilkan duplicate output setelah failure?**
   = Karena output dapat berhasil ditulis sebelum offset input di-commit. Jika terjadi failure pada jeda itu, event dibaca dan diproses ulang setelah consumer berjalan kembali, sehingga output-nya dapat terduplikasi.
b. Mengapa Exactly-once dapat menghindari duplicate committed output pada **eksperimen ini?**
   = Karena Kafka menggabungkan penulisan output dan commit offset input dalam satu transaksi. Jika transaksi gagal sebelum commit, output transaksi tersebut tidak terlihat oleh consumer read_committed, lalu event dapat diproses ulang tanpa menghasilkan duplikat committed.
c. **Apa perbedaan hasil yang Anda amati antara kedua mode tersebut?**
   = Pada pengujian ini, At-least-once menghasilkan 11 output dari 10 event unik, sehingga ada 1 duplikat. Exactly-once menghasilkan 10 output committed untuk 10 event unik dan tidak ada duplikat.

9. Evidence Chain

Sudah tertera lengkap dalam laporan ini

10. Checkpoint

a. Apa fungsi partition pada Kafka?
   = Partition membagi topic menjadi beberapa bagian agar data dapat disimpan dan diproses secara paralel. Setiap partition memiliki urutan record dan offset-nya sendiri.

b. Mengapa beberapa consumer dalam consumer group yang sama dapat menerima partition yang berbeda?
   = Kafka membagi partition topic di antara consumer yang menjadi anggota group yang sama. Dengan begitu, beberapa consumer dapat memproses data secara paralel tanpa masing-masing membaca semua partition yang sama.

c. Apa fungsi committed offset?
   = Committed offset mencatat posisi konsumsi yang telah disimpan untuk sebuah consumer group pada partition tertentu. Saat consumer berjalan kembali dengan group yang sama, konsumsi dapat dilanjutkan dari posisi tersebut.

d. Apa yang terjadi pada partition assignment ketika salah satu consumer dalam group mengalami failure?

= Kafka mendeteksi perubahan keanggotaan group dan melakukan rebalance. Partition yang sebelumnya ditangani consumer yang gagal dapat diberikan kepada consumer group lain yang masih aktif, dan event setelah offset terakhir yang di-commit mungkin dibaca ulang.

e. Berdasarkan hasil eksperimen, apa perbedaan At-least-once dan Exactly-once?
= Pada eksperimen ini, At-least-once menghasilkan 11 output untuk 10 event unik karena event yang output-nya sudah ditulis tetapi offset-nya belum di-commit dapat diproses ulang. Exactly-once menghasilkan 10 output committed untuk 10 event unik tanpa duplikat, karena output dan offset dikelola dalam satu transaksi Kafka. Ini berlaku pada alur transaksi Kafka yang diuji, bukan jaminan exactly-once untuk semua sistem eksternal.

B. TANTANGAN MANDIRI

1. Skenario

Sebuah sistem Payment Event Stream menerima event pembayaran secara asynchronous melalui Apache Kafka.

Setiap event memiliki:

event_id | order_id | event_type

Contoh:

PAY-001 | ORD-01 | CREATED

PAY-002 | ORD-01 | PAID

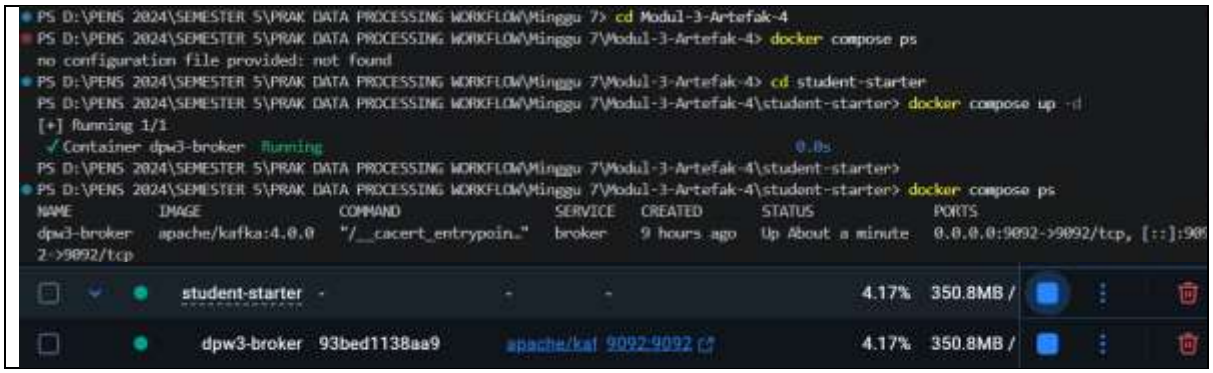
PAY-003 | ORD-02 | CREATED

PAY-004 | ORD-01 | SHIPPED

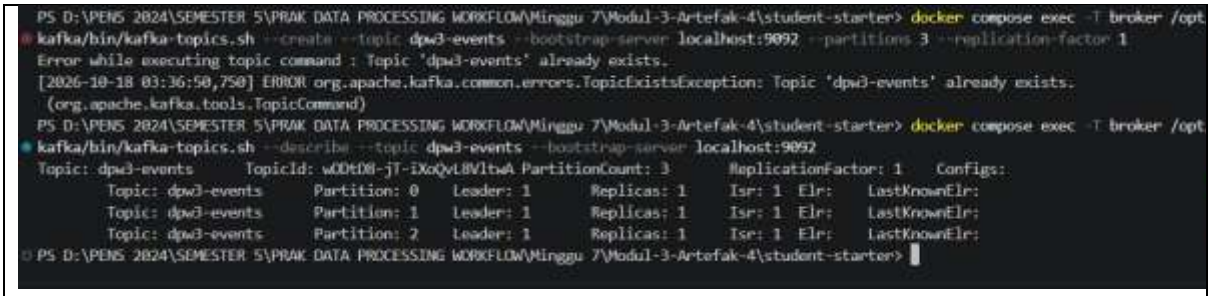
Sistem memiliki requirement: Event untuk order yang sama harus diproses secara berurutan, sedangkan event dari order yang berbeda dapat diproses secara paralel.

Pada saat processing berlangsung, salah satu consumer dapat mengalami failure.

a. Aktifkan docker broker



b. Jika topic dpw3-events belum ada, buat topic dengan tiga partition:



Ternyata sudah ada

2. Analisis Arsitektur

Konfigurasi yang dipakai:

- Partition: 3
- Consumer group: dpw3-case
- Consumer: 2
- Event key: order_id
- Offset reset: earliest
- Commit: manual, setelah processing berhasil
- Delivery: at-least-once

File starter producer.py sudah menggunakan order_id sebagai key, dan consumer.py membaca pengaturan group dari environment variable. Konfigurasi ini juga sudah dicatat di config.py.

3. Jalankan dua consumer

Langkah 1: Consumer C1 dengan failure injection pada terminal 1



Langkah 2: Consumer C2 dalam group yang sama pada terminal 2



4. Kirim event dan amati urutan

Di terminal 3

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-4> cd student-starter
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-4\student-starter> Set-Location 'D:\PENS 2024\SEMESTE
 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-4\student-starter'
>> & 'D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3\.venv\Scripts\python.exe' app\producer.py
[PRODUCER] PAY-001 order=ORD-01 key=ORD-01
[PRODUCER] PAY-002 order=ORD-02 key=ORD-02
[PRODUCER] PAY-003 order=ORD-01 key=ORD-01
[PRODUCER] PAY-004 order=ORD-03 key=ORD-03
[PRODUCER] PAY-005 order=ORD-02 key=ORD-02
[PRODUCER] PAY-006 order=ORD-01 key=ORD-01
[PRODUCER] PAY-007 order=ORD-03 key=ORD-03
[PRODUCER] PAY-008 order=ORD-02 key=ORD-02
[PRODUCER] done
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-4\student-starter>
```

Dataset berisi urutan seperti ORD-01: CREATED → PAID → SHIPPED. Karena key-nya order_id, event untuk order yang sama masuk ke partition yang sama.

5. Failure Scenario

Hasil C1: Berhenti setelah memproses 3 event

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-4\student-starter> Set-Location 'D:\PENS 2024\SEMESTE
 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-4\student-starter'
>> $env:GROUP_ID='dpw3-case'
>> $env:AUTO_OFFSET_RESET='earliest'
>> $env:FAIL_AFTER='3'
>> & 'D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3\.venv\Scripts\python.exe' -u app\consumer.py
[CONSUMER] group=dpw3-case
[PROCESS] event=PAY-002 order=ORD-02 type=CREATED partition=2 offset=9
[COMMIT] partition=2 offset=9
[PROCESS] event=PAY-005 order=ORD-02 type=PAID partition=2 offset=10
[COMMIT] partition=2 offset=10
[PROCESS] event=PAY-008 order=ORD-02 type=SHIPPED partition=2 offset=11
[FAILURE] intentional stop BEFORE commit
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-4\student-starter>
```

Hasil C2: masih berjalan

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-4\student-starter> Set-Location 'D:\PENS 2024\SEMESTE
 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-4\student-starter'
>> $env:GROUP_ID='dpw3-case'
>> $env:AUTO_OFFSET_RESET='earliest'
>> $env:FAIL_AFTER='0'
>> & 'D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3\.venv\Scripts\python.exe' -u app\consumer.py
[CONSUMER] group=dpw3-case
[PROCESS] event=PAY-001 order=ORD-01 type=CREATED partition=0 offset=9
[COMMIT] partition=0 offset=9
[PROCESS] event=PAY-003 order=ORD-01 type=PAID partition=0 offset=10
[COMMIT] partition=0 offset=10
[PROCESS] event=PAY-004 order=ORD-03 type=CREATED partition=1 offset=6
[COMMIT] partition=1 offset=6
[PROCESS] event=PAY-006 order=ORD-01 type=SHIPPED partition=0 offset=11
[COMMIT] partition=0 offset=11
[PROCESS] event=PAY-007 order=ORD-03 type=PAID partition=1 offset=7
[COMMIT] partition=1 offset=7
[PROCESS] event=PAY-008 order=ORD-02 type=SHIPPED partition=2 offset=11
[COMMIT] partition=2 offset=11
```

Periksa grup dari folder student-starter

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-4\student-starter> docker compose exec -T broker /opt/
kafka/bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --group dpw3-case

GROUP           TOPIC          PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID
HOST            CLIENT-ID
dpw3-case       dpw3-events    0          12              12              0               rdkafka-66cf39fe-eb8e-45ab-b5a8-e44ab08a6ae4
/172.23.0.1     rdkafka
dpw3-case       dpw3-events    1          8               8               0               rdkafka-66cf39fe-eb8e-45ab-b5a8-e44ab08a6ae4
/172.23.0.1     rdkafka
dpw3-case       dpw3-events    2          12              12              0               rdkafka-66cf39fe-eb8e-45ab-b5a8-e44ab08a6ae4
/172.23.0.1     rdkafka
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-4\student-starter>
```

Uji pemulihan

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-4\student-starter> Set-Location 'D:\PENS 2024\SEMESTE
5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-4\student-starter'
>> $env:GROUP_ID='dpw3-case'
>> $env:AUTO_OFFSET_RESET='earliest'
>> $env:FAIL_AFTER='0'
>> & 'D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3\.venv\Scripts\python.exe' -u app\consumer.py
[CONSUMER] group=dpw3-case
[PROCESS] event=PAY-001 order=ORD-01 type=CREATED partition=0 offset=12
[COMMIT] partition=0 offset=12
[PROCESS] event=PAY-003 order=ORD-01 type=PAID partition=0 offset=13
[COMMIT] partition=0 offset=13
[PROCESS] event=PAY-004 order=ORD-03 type=CREATED partition=1 offset=8
[COMMIT] partition=1 offset=8
[PROCESS] event=PAY-006 order=ORD-01 type=SHIPPED partition=0 offset=14
[COMMIT] partition=0 offset=14
[PROCESS] event=PAY-007 order=ORD-03 type=PAID partition=1 offset=9
[COMMIT] partition=1 offset=9
```

Data yang dikirim untuk memastikan consumer yang aktif melanjutkan pemrosesan

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-4\student-starter> & 'D:\PENS 2024\SEMESTER 5\PRAK DAT
A PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-3\.venv\Scripts\python.exe' app\producer.py
[PRODUCER] PAY-001 order=ORD-01 key=ORD-01
[PRODUCER] PAY-002 order=ORD-02 key=ORD-02
[PRODUCER] PAY-003 order=ORD-01 key=ORD-01
[PRODUCER] PAY-004 order=ORD-03 key=ORD-03
[PRODUCER] PAY-005 order=ORD-02 key=ORD-02
[PRODUCER] PAY-006 order=ORD-01 key=ORD-01
[PRODUCER] PAY-007 order=ORD-03 key=ORD-03
[PRODUCER] PAY-008 order=ORD-02 key=ORD-02
[PRODUCER] done
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 7\Modul-3-Artefak-4\student-starter>
```

- **Action:** Menjalankan dua consumer dengan group dpw3-case. Consumer C1 dikonfigurasi gagal setelah tiga event, lalu producer mengirim delapan event.
- **Observation:** C1 memproses PAY-002, PAY-005, lalu gagal pada PAY-008 di partition 2, offset 2, sebelum commit. C2 kemudian memproses PAY-008 pada partition dan offset yang sama, lalu melakukan commit.
- **Diagnosis:** Karena C1 gagal sebelum commit PAY-008, offset event itu belum tersimpan sebagai selesai. Setelah rebalance, C2 membaca ulang dan memproses event tersebut.
- **Fix:** Menjalankan kembali consumer dengan group dpw3-case yang sama agar Kafka dapat melanjutkan konsumsi menggunakan committed offset group tersebut.
- **Verification:** Consumer yang dipulihkan memproses event dan melakukan commit. Pemeriksaan akhir menunjukkan CURRENT-OFFSET 9, 6, dan 9 pada partition 0, 1, dan 2, dengan LAG 0.